

-0 / +18 lines of 176 · 2 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (2 code line(s) added, 0 removed) · restructuring level 0

BEFORE	AFTER	file
132 } 133 134 static real_t kernel_s211(void) 135 { 136 for (int nl = 0; nl < R; nl++) {	132 } 133 134 static real_t kernel_s211(void) 135 { 136 for (int nl = 0; nl < R; nl++) { 137 /* Loop distribution: split the inner loop into two independent loops. 138 * Loop A computes a[i] using b values from the previous iteration. 139 * Loop B computes new b[i] values for the next iteration. 140 * No loop-carried dependences within each loop, so both can be parall 141 */ 142 143 /* Loop A: compute a[i] = b[i - 1] + c[i] * d[i] for all i 144 * Each iteration writes to a unique a[i], no loop-carried dependence. 145 */ 146 #pragma omp parallel for 147 for (int i = 1; i < LEN_1D-1; i++) { 148 a[i] = b[i - 1] + c[i] * d[i]; 149 } 150 151 /* Loop B: compute b[i] = b[i + 1] - e[i] * d[i] for all i 152 * Each iteration writes to a unique b[i], no loop-carried dependence. 153 */ 154 #pragma omp parallel for 155 for (int i = 1; i < LEN_1D-1; i++) { 156 b[i] = b[i + 1] - e[i] * d[i]; 157 } 158 159 pb_mix(nl); 160 } 161 return (real_t)0; 162 } 163	
137 for (int i = 1; i < LEN_1D-1; i++) { 138 a[i] = b[i - 1] + c[i] * d[i];		
139 b[i] = b[i + 1] - e[i] * d[i]; 140 }		
141 pb_mix(nl); 142 } 143 return (real_t)0; 144 } 145		